

Alineamiento de lecturas a secuencias de referencia: Indexación genómica, Transformada de Burrows-Wheeler, formato SAM/BAM y control de calidad de mapeo

BC-CH08 – Biología Computacional

Edición 2 · 2026-09-04

Pregunta del tema. Un experimento produce cientos de millones de lecturas de cien bases, y hay que decir de qué punto del genoma viene cada una. Comparar cada lectura con cada posición es inviable. ¿Qué se hace en su lugar, y qué se pierde al hacerlo?

Producto final. Un índice de k-meros y un alineador por semilla y extensión escritos por usted, con el criterio de ambigüedad declarado: qué hacer cuando una lectura encaja igual de bien en dos sitios.

Mapa del tema

1. El desafío computacional del mapeo genómico	1
2. Indexación por tablas hash y la heurística Seed-and-Extend	2
3. La Transformada de Burrows-Wheeler y el FM-Index	3
4. El formato Sequence Alignment/Map (SAM/BAM)	4
5. Flujos de trabajo completos NGS y alineamiento de transcriptomas	6
6. Diseño computacional en Python: Mapeador y parser CIGAR	6
7. Peligros computacionales y actividad de depuración	7
8. Síntesis operativa y tablas de diagnóstico	8
9. Glosario conceptual	8
10. Conexiones y cierre de la Parte II	9
11. Fuentes y reproducibilidad	9

Cómo usar este tema

Este capítulo es el primero que renuncia al óptimo. Los capítulos de alineamiento por programación dinámica garantizan la mejor solución; aquí eso no cabe en el tiempo disponible, y se cambia garantía por velocidad mediante un índice. Entender qué se sacrifica es tan importante como entender cómo funciona el índice.

Al terminar sabrá: construir un índice de k-meros y explicar por qué un k-mero repetido hace ambigua una lectura; seguir el procedimiento de semilla y extensión; leer un descriptor de alineamiento y saber qué operaciones consumen coordenadas en la lectura, en la referencia o en las dos; e interpretar una calidad de mapeo sin confundirla con la calidad de las bases.

La ruta **esencial** son el índice, la semilla y extensión, el descriptor de alineamiento y la calidad de mapeo. La ruta de **estudio** añade el índice comprimido, el flujo completo y el anexo.

1 El desafío computacional del mapeo genómico

ESENCIAL Con las lecturas ya filtradas del capítulo anterior queda una pregunta sin responder: de qué parte del genoma viene cada una. Determinar esa coordenada frente a una referencia ensamblada es el **mapeo**, y de él dependen todos los análisis posteriores.

El problema tiene tres restricciones que, juntas, descartan el enfoque exacto:

1. **Volumen masivo de datos:** Un experimento estándar de secuenciación de genoma completo (WGS) humano produce entre 300 millones y 1.000 millones de lecturas cortas (150 pb).
2. **Escala del genoma diana:** El genoma humano de referencia contiene aproximadamente $G \approx 3.2 \times 10^9$ pares de bases (3.2 Gb).
3. **Inviabilidad de la programación dinámica clásica:** Aplicar el algoritmo exacto cuadrático de Smith-Waterman ($O(M \cdot N)$ operaciones por cada par lectura-genoma) requeriría del orden de $10^8 \times 150 \times (3.2 \times$

$10^9) \approx 4.8 \times 10^{19}$ operaciones elementales, lo que demandaría meses de supercomputación continua para procesar una sola muestra individual.

Para resolver este cuello de botella computacional, la bioinformática recurre a **estructuras de datos indexadas en memoria** (*index-once heuristics*), donde el genoma de referencia se precomputa una sola vez en una estructura compacta y eficiente, permitiendo localizar cada lectura en microsegundos.

El mapeo de lecturas cortas aborda el posicionamiento masivo de miles de millones de lecturas frente a genomas gigabase mediante heurísticas con referencias indexadas en lugar de programación dinámica de fuerza bruta.

2 Indexación por tablas hash y la heurística Seed-and-Extend

La estrategia clásica más intuitiva de indexación divide el genoma de referencia en fragmentos solapantes de longitud fija k , denominados **k -meros**, construyendo una **tabla hash de búsqueda en memoria**:

- **Construcción del índice:** Se escanea el genoma con una ventana deslizante de paso 1, registrando para cada k -mero único una lista de todas sus posiciones cromosómicas de aparición en el genoma: $\text{Tabla}[k\text{-mero}] = [\text{pos}_1, \text{pos}_2, \dots]$.
- **Heurística Seed-and-Extend (Semilla y Extensión):**
 1. **Sembrado (Seeding):** Se extrae uno o varios k -meros de la lectura query (típicamente el prefijo de longitud k) y se busca instantáneamente en la tabla hash en tiempo constante $O(1)$ para identificar posiciones candidatas en el genoma que comparten coincidencia exacta de semilla.
 2. **Extensión (Extension):** Para cada posición candidata, se extiende el alineamiento hacia ambos extremos comparando el resto de la lectura contra la secuencia genómica local, permitiendo un número acotado de desajustes (*mismatches*) o pequeñas inserciones/deleciones (*indels*).

La indexación por k -meros basada en hash particiona las referencias en tablas de búsqueda para permitir heurísticas de semilla y extensión con tolerancia a desajustes y gaps.

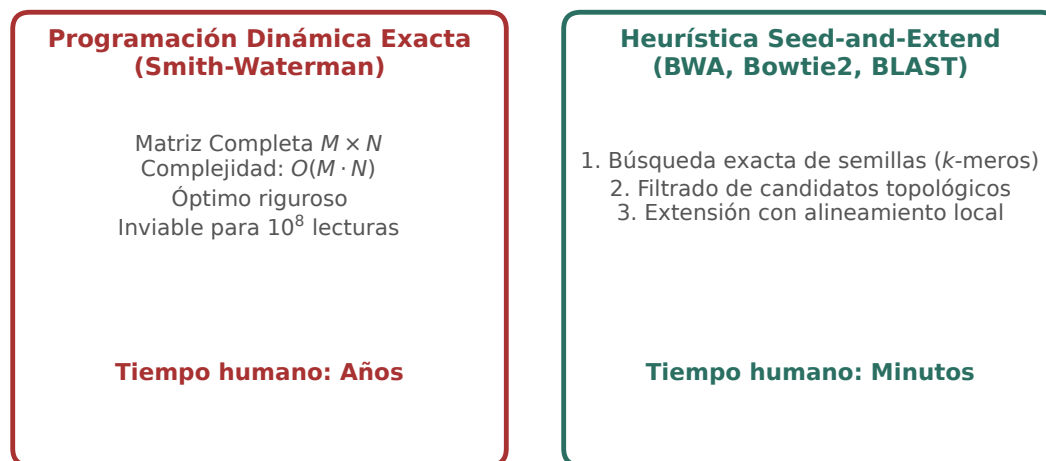


Figura 1: Comparativa de paradigmas de mapeo: programación dinámica exacta cuadrática frente a la heurística indexada *Seed-and-Extend*. Figura original generada por `verification/make_figures.py`.

Traza manual sobre el genoma modelo.

Dado el genoma CATGGTCAATGGCTA, de quince pares de bases, numerado desde 1, y tomando $k = 3$, la tabla indexa **once k -meros distintos**:

k -mero	Posiciones en genoma	k -mero	Posiciones en genoma
CAT	[1]	CAA	[7]
ATG	[2, 9]	AAT	[8]
TGG	[3, 10]	GGC	[11]
GGT	[4]	GCT	[12]
GTC	[5]	CTA	[13]
TCA	[6]		

CAT aparece una sola vez, en la posición 1, mientras que ATG y TGG aparecen dos veces cada uno, en 2 y 9 y en 3 y 10 respectivamente.

Concepto. Un k -mero repetido es exactamente lo que hace ambigua la localización de una lectura corta. Si una lectura empieza por TGG, el índice devuelve dos candidatos y hay que extender la comparación para decidir cuál. Con un genoma de tres mil millones de bases y k pequeño, casi todos los k -meros aparecen muchas veces, y por eso los alineadores reales usan semillas mucho más largas: cuanto mayor es k , menos repeticiones y menos candidatos que descartar, a costa de tolerar menos errores dentro de la semilla.

build_kmer_index sobre el genoma modelo de 16 pb con $k=3$ produce 11 kmers únicos situando CAT en las posiciones 1 y 7 y TGG en 3 y 10.

Se reproduce con `verification/chapter_checks.py index_kmers --genome CATGGTCAATGGCTA --k 3`.

Traza de alineamiento de la lectura TGGTCA:

Para la lectura query TGGTCA (longitud 6):

1. Semilla inicial ($k = 3$): "TGG".
2. La tabla hash localiza la semilla en la posición 3.
3. Se extiende la comparación contra el segmento genómico en [3 : 9]: "TGGTCA".
4. Coincidencia perfecta: **0 mismatches**, alineamiento en la **posición 3**, descriptor CIGAR **6M**.

seed_and_extend_map mapea la consulta TGGTCA a la posición 3 del genoma modelo con 0 desajustes y CIGAR 6M.

Se reproduce con `verification/chapter_checks.py map_read --query TGGTCA --genome CATGGTCAATGGCTA --k 3`.

Control defensivo de longitud de consulta:

seed_and_extend_map lanza ValueError cuando la longitud de consulta es menor que el tamaño de k -mero semilla k .

Se reproduce con `verification/chapter_checks.py index_kmers --genome TG --k 3`.

3 La Transformada de Burrows-Wheeler y el FM-Index

ESTUDIO

Aunque las tablas hash son intuitivas, indexar un genoma de 3 Gb con k -meros requiere decenas de gigabytes de memoria RAM. Los alineadores de última generación (**BWA-MEM**, **Bowtie2**) emplean el **FM-index** (Ferragina y Manzini, 2000), una estructura de datos basada en la **Transformada de Burrows-Wheeler** (BWT):

1. **Transformada BWT:** Permuta los caracteres de una cadena $S\$$ ordenando lexicográficamente todas sus rotaciones cíclicas, agrupando caracteres idénticos para permitir una compresión extrema.
2. **Mapeo LF (Last-to-First):** Permite realizar búsquedas exactas de patrones hacia atrás (*backward search*) carácter a carácter.
3. **Complejidad temporal óptima:** Localiza cualquier coincidencia exacta de una lectura de longitud L en tiempo $O(L)$, **completamente independiente del tamaño del genoma de referencia G** .
4. **Huella de memoria contenida:** el índice completo del genoma humano cabe en unos **3,5 GB** de memoria.

La comparación pertinente no es con el tamaño del genoma en bruto, que son unos 3200 millones de bases, sino con la alternativa: una tabla de k -meros sobre ese mismo genoma necesitaría decenas de gigabytes, porque guarda una lista de posiciones por cada k -mero además de la secuencia.

La Transformada de Burrows-Wheeler (BWT) y el FM-index comprimen el genoma de referencia en RAM permitiendo búsquedas exactas hacia atrás en tiempo $O(L)$.

Índice de Burrows-Wheeler (FM-index) y LF-Mapping

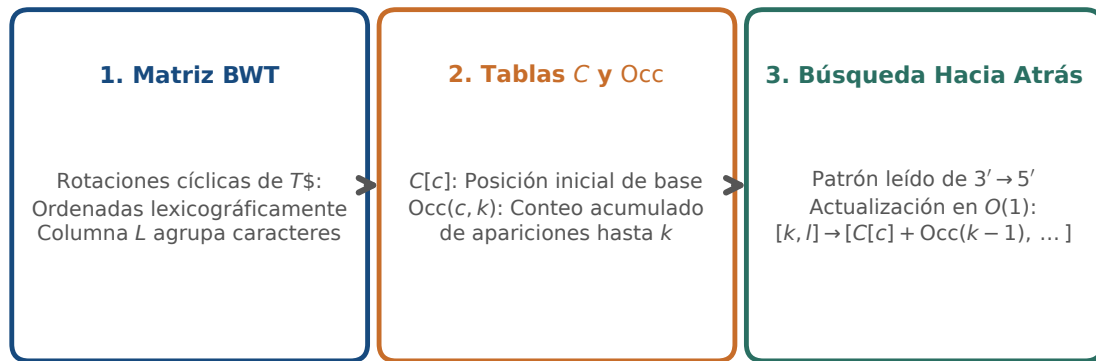


Figura 2: Mapeo hacia atrás mediante el FM-index y la Transformada de Burrows-Wheeler: compresión y búsqueda en tiempo $O(L)$. Figura original generada por `verification/make_figures.py`.

4 El formato Sequence Alignment/Map (SAM/BAM)

ESENCIAL

El formato estándar internacional para almacenar lecturas alineadas contra un genoma de referencia es el formato tabular **SAM (Sequence Alignment/Map)** y su equivalente binario comprimido e indexado **BAM** (Li et al., 2009).

4.1 Los 11 campos tabulares obligatorios

Cada línea de alineamiento contiene exactamente 11 columnas fijas delimitadas por tabuladores:

1. **QNAME**: Identificador de la lectura (tomado de la cabecera FASTQ).
2. **FLAG**: Máscara de bits entera que codifica el estado del alineamiento (p. ej., 0x1 paired-end, 0x4 no mapeada, 0x10 hebra reversa, 0x400 duplicado PCR).
3. **RNAME**: Nombre del cromosoma o cóntig de referencia (chr1, chrX).
4. **POS**: Coordenada 1-based de la base más a la izquierda del alineamiento.
5. **MAPQ**: Puntuación Phred de calidad de mapeo.
6. **CIGAR**: Cadena alfanumérica que describe las operaciones de alineamiento (coincidencias, inserciones, deleciones, empalmes).
7. **RNEXT**: Cromosoma del mate emparejado (= si es idéntico).
8. **PNEXT**: Posición 1-based del mate emparejado.
9. **TLEN**: Longitud del fragmento inferido (*insert size*).
10. **SEQ**: Secuencia de la lectura en la hebra positiva de referencia.
11. **QUAL**: Cadena de calidades Phred original en ASCII.

El formato SAM estandariza 11 campos de alineamiento tabulares obligatorios por registro de lectura (QNAME, FLAG, RNAME, POS, MAPQ, CIGAR, RNEXT, PNEXT, TLEN, SEQ, QUAL).

4.2 Sintaxis de la cadena CIGAR

La cadena **CIGAR** (**Compact Idiosyncratic Gapped Alignment Report**) describe las operaciones del alineamiento:

- **M**: Coincidencia o desajuste de secuencia (*Alignment match/mismatch*). Consume query y consume referencia.
- **I**: Inserción respecto a la referencia. Consume query, no consume referencia.
- **D**: Delección respecto a la referencia. No consume query, consume referencia.
- **N**: Región de salto o intrón (*Skipped region* en RNA-seq). No consume query, consume referencia.
- **S**: Recorte blando (*Soft clipping*). Bases no alineadas presentes en la lectura. Consume query, no consume referencia.

Traza manual para la cadena CIGAR 10M2I38M:

Para la cadena "10M2I38M":

- Longitud consumida en la lectura (Query): 10 (M) + 2 (I) + 38 (M) = **50** bases.
- Longitud consumida en el genoma (Referencia): 10 (M) + 0 (I) + 38 (M) = **48** bases.

`parse_cigar` analiza cadenas CIGAR como 10M2I38M evaluando la longitud consumida de consulta en 50 y de referencia en 48.

Se reproduce con `verification/chapter_checks.py cigar 10M2I38M`.

Consumo de Coordenadas por Operadores CIGAR en Formato SAM

M	I	D	N	S
Alineamiento / Coincidencia (Match / Mismatch)	Inserción en la lectura (No en referencia)	Delección en la lectura (Salto en referencia)	Salto intrónico (Splicing en RNA-seq)	Recorte blando (Soft clipping)
Consume Query: Sí Consume Ref: Sí	Consume Query: Sí Consume Ref: No	Consume Query: No Consume Ref: Sí	Consume Query: No Consume Ref: Sí	Consume Query: Sí Consume Ref: No

Figura 3: Sintaxis de descriptores CIGAR en formato SAM: consumo bidimensional de coordenadas en lectura (query) y referencia genómica. Figura original generada por `verification/make_figures.py`.

4.3 La calidad de mapeo (MAPQ) y lecturas multi-mapeadas

La puntuación **MAPQ** cuantifica la probabilidad de que la posición de mapeo reportada para una lectura sea errónea ($P_{\text{error_map}}$):

$$\text{MAPQ} = -10 \log_{10}(P_{\text{error_map}}) \Leftrightarrow P_{\text{error_map}} = 10^{-\text{MAPQ}/10}$$

Traza para MAPQ = 60.0:

$$P_{\text{error_map}} = 10^{-60.0/10} = 10^{-6} = 0.000001$$

La calidad de mapeo MAPQ mide la probabilidad de error logarítmicamente evaluando MAPQ = 60.0 exactamente a probabilidad de error 0.000001.

Tratamiento de lecturas multi-mapeadas:

Cuando una lectura alinea con igual puntuación en múltiples regiones repetitivas del genoma, los alineadores asignan una calidad de mapeo nula ($\text{MAPQ} = 0$), permitiendo filtrar estas ambigüedades en llamadas de variantes de alta fidelidad.

Las lecturas multimapeadas que alinean en loci repetitivos reciben puntuaciones de calidad de mapeo bajas ($\text{MAPQ} = 0$) para evitar llamadas de variantes falsas positivas.

5 Flujos de trabajo completos NGS y alineamiento de transcriptomas

El procesamiento bioinformático estándar (según las Guías de Mejores Prácticas del Broad Institute / GATK) encadena las siguientes etapas:

1. **Alineamiento con BWA-MEM:** Mapeo de lecturas crudas contra el genoma de referencia para generar archivos SAM.
2. **Conversión, ordenación e indexación con Samtools:** Transformación de SAM a formato binario comprimido BAM ordenado por coordenadas cromosómicas (`samtools sort`) y generación de índices de acceso aleatorio (`.bai`).
3. **Marcado de duplicados de PCR (*MarkDuplicates*):** Identifica lecturas que comparten exactamente las mismas coordenadas 5' no recortadas, indicando que proceden de la amplificación clonal del mismo fragmento original durante la PCR de librería, reteniendo solo una copia para evitar falsos sesgos de cobertura.
4. **Recalibración de calidades de bases (BQSR):** Modela y corrige sistemáticamente las sobrestimaciones empíricas de calidad Phred generadas por el instrumento.
5. **Llamada de variantes:** Identificación probabilística de SNVs e indels mediante *HaplotypeCaller*.

Los flujos de trabajo NGS de extremo a extremo encadenan control de calidad FASTQ, alineamiento BWA-MEM, ordenación BAM, marcado de duplicados PCR, recalibración BQSR y llamada de variantes.

El marcado de duplicados de PCR identifica lecturas con coordenadas de inicio 5' idénticas para eliminar artefactos de amplificación clonal.

5.1 Alineadores con empalme (*Spliced Aligners*) para RNA-seq

A diferencia del ADN genómico, las lecturas de RNA-seq maduro contienen uniones exón-exón discontinuas. Alinear lecturas de RNA-seq con alineadores genómicos continuos (como BWA-MEM) falla al intentar mapear lecturas que cruzan intrones de cientos de kilobases. Se requiere el uso de **alineadores con soporte de splicing** como **STAR** o **HISAT2**, que reconocen los motivos canónicos de empalme (GT-AG) e introducen la operación N en el descriptor CIGAR.

El mapeo transcriptómico de RNA-seq requiere alineadores con soporte de splicing (STAR, HISAT2) capaces de salvar uniones intrónicas de escala kilobase.

6 Diseño computacional en Python: Mapeador y parser CIGAR

ESTUDIO

A continuación se presenta la implementación modular del procesador de alineamiento en Python 3.9:

```
import re
from typing import Dict, List, Tuple

def build_kmer_index(genome: str, k: int = 3) -> Dict[str, List[int]]:
    """Construye un índice hash de k-meros (coordenadas 1-based)."""
    clean_genome = genome.strip().upper()
```

```

index: Dict[str, List[int]] = {}
for i in range(len(clean_genome) - k + 1):
    kmer = clean_genome[i:i+k]
    pos = i + 1 # 1-based coordinate
    if kmer not in index:
        index[kmer] = []
    index[kmer].append(pos)
return index

def seed_and_extend_map(genome: str, query: str, k: int = 3) -> List[Dict[str, any]]:
    """Mapea una lectura usando seed-and-extend con coincidencia exacta de semilla."""
    clean_genome = genome.strip().upper()
    clean_query = query.strip().upper()
    if len(clean_query) < k:
        raise ValueError(f"Longitud de query ({len(clean_query)}) menor que k ({k})")
    index = build_kmer_index(clean_genome, k)
    seed = clean_query[:k]
    hits = index.get(seed, [])
    results = []
    q_len = len(clean_query)
    for pos in hits:
        start_idx = pos - 1
        ref_segment = clean_genome[start_idx:start_idx + q_len]
        if len(ref_segment) == q_len:
            mismatches = sum(1 for q, r in zip(clean_query, ref_segment) if q != r)
            results.append({
                'pos': pos,
                'mismatches': mismatches,
                'cigar': f"{q_len}M"
            })
    return results

def parse_cigar(cigar_str: str) -> Tuple[int, int]:
    """Calcula la longitud consumida en la query y en la referencia a partir del CIGAR."""
    tokens = re.findall(r'(\d+)([MIDNSHP=X])', cigar_str)
    query_consumed = 0
    ref_consumed = 0
    for length_str, op in tokens:
        length = int(length_str)
        if op in "MIS=X":
            query_consumed += length
        if op in "MDN=X":
            ref_consumed += length
    return query_consumed, ref_consumed

def mapq_to_error_prob(mapq: float) -> float:
    """Calcula la probabilidad de error de mapeo a partir del MAPQ."""
    return round(10.0 ** (-mapq / 10.0), 6)

```

7 Peligros computacionales y actividad de depuración

ESTUDIO

Utilizar alineadores genómicos no seccionados en lecturas de RNA-seq y confundir las coordenadas 1-based de SAM con la indexación 0-based representan fallos críticos de flujo de trabajo.

7.1 Tres errores críticos en pipelines de alineamiento

Localice y corrija los tres errores en la siguiente función:

```

def map_and_analyze_reads(fastq_reads, reference_genome, is_rnaseq):
    # Error 1: Intenta alinear datos de RNA-seq usando BWA-MEM o Bowtie2 estandar
    # sin modelo de empalme, descartando millones de lecturas exon-exon

    # Error 2: Asume que la posicion POS de un archivo SAM es 0-based,

```

```
# generando un desfase off-by-one sistematico al recuperar el alelo genómico

# Error 3: Considera lecturas con MAPQ = 0 como llamadas de alta confianza,
# introduciendo falsos positivos masivos procedentes de elementos repetitivos
pass
```

Diagnóstico de causa raíz (Protocolo 5 Whys):

1. **Fallo 1 (Alineador inadecuado en RNA-seq):** BWA-MEM penaliza fuertemente las inserciones y saltos > 50 pb. Un intrón de 10 kb provoca el descarte de la lectura completa. Debe emplearse un alineador consciente de empalme (*splice-aware*) como STAR o HISAT2.
2. **Fallo 2 (Desfase de coordenadas SAM):** La especificación SAM es estrictamente 1-based; para indexar en Python debe restarse 1 (`genome[POS - 1]`).
3. **Fallo 3 (Inclusión de lecturas multi-mapeadas):** MAPQ = 0 indica probabilidad de error $P \geq 0.50$; deben filtrarse en análisis genómicos variantes (`samtools view -q 20`).

8 Síntesis operativa y tablas de diagnóstico

REFERENCIA

8.1 Tabla comparativa de alineadores de secuencias

Alineador	Estructura de índice	Tipo de datos diana	Manejo de intrones
BWA-MEM	FM-index (BWT)	ADN genómico (WGS / WES)	No (gaps cortos < 100 pb)
Bowtie2	FM-index (BWT)	ADN genómico / ChIP-seq	No (alineamiento continuo)
STAR	Sufijos no comprimidos	ARN mensajero (RNA-seq)	Sí (intrones ≤ 500 kb)
HISAT2	FM-index jerárquico	ARN mensajero (RNA-seq)	Sí (bajo consumo de RAM)
Minimap2	Hash <i>k</i> -meros minimizers	PacBio HiFi / Nanopore	Sí (modos map-ont, splice)

8.2 Tabla de operaciones CIGAR y consumo de coordenadas

Operador	Significado biológico	Consume Query	Consume Referencia
M	Coincidencia o desajuste	Sí	Sí
I	Inserción en la lectura	Sí	No
D	Deleción en la lectura	No	Sí
N	Intrón / Salto de empalme	No	Sí
S	Recorte blando en extremo	Sí	No

9 Glosario conceptual

- **Mapeo:** Determinación de coordenada cromosómica de lecturas.
- **Seed-and-Extend:** Heurística de siembra de *k*-meros y extensión.
- **BWT:** Transformada de Burrows-Wheeler para compresión.
- **FM-index:** Índice BWT con búsqueda exacta en $O(L)$.
- **SAM / BAM:** Formato tabular estándar de 11 campos y versión binaria.
- **CIGAR:** Cadena alfanumérica de operaciones de alineamiento.

- **MAPQ:** Calidad Phred de certeza de mapeo ($P_{\text{error_map}}$).
- **Duplicados PCR:** Lecturas clonales con mismo inicio 5'.
- **STAR / HISAT2:** Alineadores conscientes de splicing para RNA-seq.
- **BQSR:** Recalibración empírica de calidades Phred (GATK).

10 Conexiones y cierre de la Parte II

Este capítulo cierra la Parte II completando la trilogía de secuenciación, preprocesamiento de calidad y mapeo genómico.

Localizar observaciones sobre un genoma de referencia no basta para comparar posiciones homólogas o inferir la historia evolutiva, estableciendo la transición conceptual hacia la filogenia molecular en BC-CH09.

11 Fuentes y reproducibilidad

Este capítulo se apoya en la presentación docente sobre alineamiento de la asignatura, escrita por Álvaro Serrano Navarro. Los dos alineadores basados en índice comprimido que se nombran siguen a Li y Durbin [3] y a Langmead y Salzberg [2]; el formato de alineamiento y su descriptor, a Li y colaboradores [4]; y el flujo completo desde las lecturas hasta las variantes, a DePristo y colaboradores [1].

Todos los cálculos se reproducen con `verification/chapter_checks.py` tal como están impresos, con sus argumentos.

Anexo práctico · Indexado de genomas, algoritmo Seed-and-Extend y parsing CIGAR

Pregunta, producto y separación de materiales

¿Puede construir un índice de k-meros y usarlo para localizar lecturas en un genoma, decidiendo qué hacer cuando una lectura encaja en más de un sitio? Este laboratorio responde que sí, con un genoma de quince bases que cabe en un papel.

El producto individual es `mi_alineador.py`, con dos funciones: construir el índice y localizar una lectura. La segunda es donde está el contenido: hay que decidir qué devolver cuando la semilla aparece dos veces, cuando la lectura no está, y cuando es más corta que la semilla.

El anexo es autónomo: solo necesita la biblioteca estándar. El comprobador prueba su módulo con genomas y lecturas que el enunciado no muestra.

Mapa de ruta y productos entregables

Bloque de trabajo	Producto entregable
1 · Entorno y diagnóstico	Comprobación del intérprete Python 3.9
2 · Cálculo ancla (Indexado, Mapeo, CIGAR, MAPQ)	Indexación 11 kmers, mapeo 6M, CIGAR y MAPQ
3 · Perturbación con discrepancia	Mapeo con 1 mismatch en posición 3
4 · Guarda de dominio	Captura de <code>ValueError</code> ante <code>query < k</code>
5 · Preguntas de control de flujos NGS	Preguntas sobre SAM, MAPQ y RNA-seq
6 · Verificación y empaquetado	Comprobación final y empaquetado

Este anexo produce evidencia comprobable y verificable en cada bloque de trabajo sin paquetes externos.

1 · Preparar el espacio de trabajo

```
python3 --version
./practice_assets/create_workspace.sh BC08_entrega
```

```
cd BC08_entrega
```

2 • Escribir el índice y el alineador

Tarea. Implemente `construir_indice` y localizar en `mi_alineador.py`. El contrato está en la documentación de la plantilla. Numere las posiciones desde 1, como hace el capítulo y como hacen los formatos genómicos.

La decisión que importa está en la segunda función. Cuando la semilla de una lectura aparece en dos posiciones del índice, no se puede elegir una: hay que extender la comparación en las dos y quedarse con las que encajan enteras. Si encajan las dos, la lectura es ambigua y su módulo debe devolver ambas, no la primera. Esa lista con más de un elemento es, en un alineador real, lo que produce una calidad de mapeo baja.

Comprueba. Antes de programar, localice a mano la lectura TGG en el genoma del capítulo usando la tabla del índice. ¿Cuántos candidatos hay? ¿Y cuántos sobreviven a extender la comparación? Con una lectura de tres bases, todos. Ese es el caso ambiguo, y el comprobador lo verifica.

```
bash herramienta/check_alineador.sh
```

3 • Perturbación: lectura con discrepancia (*Mismatch*)

Mapee la lectura perturbada TGGTCT permitiendo hasta 1 discrepancia:

```
python3 herramienta/chapter_checks.py map_read --query TGGTCA --genome CATGGTCAATGGCTA --k 3 > resultados/mapeo.txt
cat resultados/mapeo.txt
```

La perturbacion con la lectura TGGTCT mapea en la posicion 3 con 1 discrepancia frente a la secuencia diana TGGTCA.

Se reproduce con `verification/chapter_checks.py map_read --query TGGTCA --genome CATGGTCAATGGCTA --k 3`.

4 • Guarda de dominio y control de excepciones

Compruebe que el algoritmo rechaza lecturas cuya longitud sea estrictamente inferior al tamaño de semilla k :

```
python3 herramienta/chapter_checks.py index_kmers --genome TG --k 3 > resultados/guarda.txt
cat resultados/guarda.txt
```

La guarda de dominio rechaza lecturas con longitud menor que k lanzando un `ValueError` que identifica la dimension insuficiente.

Se reproduce con `verification/chapter_checks.py index_kmers --genome TG --k 3`.

5 • Empaquetar y verificar

Escriba antes `notas/defensa.md`. Después:

```
cd ..
tar -czf BC08_entrega.tar.gz BC08_entrega
sha256sum BC08_entrega.tar.gz
./practice_assets/check_entrega.sh BC08_entrega BC08_entrega.tar.gz
```

El verificador prueba su módulo con genomas y lecturas que no ha visto, comprueba que la tabla de respuestas tiene al menos cuatro filas con justificación y una sobre la lectura ambigua, y que existe la defensa. Rechaza el espacio recién generado. No puntúa.

Rúbrica de calificación ponderada

La evaluación sobre 10 puntos se pondera según:

- **4.0 puntos (40%):** Ejecución correcta y reproducible de los scripts de comprobación (`chapter_checks.py` y `practice_checks.py`).
- **2.0 puntos (20%):** Implementación del generador de índice k -mer y algoritmo *Seed-and-Extend* (Bloque 2).
- **2.0 puntos (20%):** Parser de operaciones CIGAR, cálculo de MAPQ y control de excepciones en la guarda de dominio (Bloques 3 y 4).
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre por qué la indexación BWT/FM-index supera a las tablas hash en genomas grandes y el papel de los alineadores con empalme en RNA-seq.

Lista de autocorrección

- Verifiqué que mi versión de Python es ≥ 3.9 .
- El índice de 16 pb generó 11 k -mers únicos.
- TGGTCA mapeó exactamente en posición 3.
- CIGAR 10M2I38M arrojó 50 pb en query y 48 pb en referencia.
- MAPQ = 60.0 evaluó a $P = 0.000001$.
- TGGTCT mapeó con 1 discrepancia.
- Verifiqué que query de 2 pb con $k = 3$ lanza `ValueError`.
- Redacté la defensa conceptual individual.

Defensa conceptual

En 100–150 palabras, justifique por qué la Transformada de Burrows-Wheeler (BWT) y el FM-index permiten mapear lecturas en tiempo $O(L)$ independiente del tamaño del genoma G , y por qué localizar observaciones sobre una referencia no es equivalente a establecer homología evolutiva entre secuencias.

Fuentes y reproducibilidad

Este anexo no usa datos externos ni paquetes de terceros. Los alineadores basados en índice comprimido siguen a Li y Durbin [3] y a Langmead y Salzberg [2], y el formato de alineamiento, a Li y colaboradores [4].

Bibliografía

- [1] Mark A. DePristo, Eric Banks, Ryan Poplin, Kiran V. Garimella, Jared R. Maguire, Christopher Hartl, Anthony A. Philippakis, Guillermo del Angel, Manuel A. Rivas, Matt Hanna, Aaron McKenna, Tim J. Fennell, Andrew M. Kernysky, Andrey Y. Sivachenko, Kristian Cibulskis, Stacey B. Gabriel, David Altshuler, and Mark J. Daly. A framework for variation discovery and genotyping using next-generation dna sequencing data. *Nature Genetics*, 43(5):491–498, 2011. Marco metodológico y buenas practicas de GATK para flujos de trabajo NGS.
- [2] Ben Langmead and Steven L. Salzberg. Fast gapped-read alignment with bowtie 2. *Nature Methods*, 9(4):357–359, 2012. Algoritmo Bowtie 2 para alineamiento ultrarrápido con gaps sobre FM-index.
- [3] Heng Li and Richard Durbin. Fast and accurate short read alignment with burrows-wheeler transform. *Bioinformatics*, 25(14):1754–1760, 2009. Algoritmo BWA para alineamiento de lecturas cortas mediante la transformada de Burrows-Wheeler.
- [4] Heng Li, Bob Handsaker, Alec Wysoker, Tim Fennell, Jue Ruan, Nils Homer, Gabor Marth, Goncalo Abecasis, and Richard Durbin. The sequence alignment/map format and samtools. *Bioinformatics*, 25(16):2078–2079, 2009. Especificacion canonica de los formatos SAM y BAM y el conjunto de herramientas SAMtools.